Objectifs de la feuille

- Améliorations ergonomiques
- Histoire de performances
- Barre de recherche
- Et notre table de jointure alors ?

Améliorations ergonomiques

Les vues qui listent les produits (ProduitListView), les catégories de produits (CategorieListView), les statuts des produis (StatutListView) et les rayons (RayonListView) ressemblent enfin à des vues dignes de ce nom avec la possibilité de créer, éditer, modifier voire supprimer.

En revanche, je ne sais pas vous, mais mes vues restent pauvres en terme d'informations ramenées à l'écran. Concernant les catégories, les statuts et les rayons, seul le nom est visible. Heureusement que ma vue qui listent les produits est un peu plus fournie avec les nom, prix HT, catégorie, statut et la date de création.

Je vous propose de travailler sur les vues les moins chargées. Commençons par la vue qui liste les catégories CategorieListView. Je veux afficher le nombre de produits qui compte une catégorie. Très simplement, nous allons modifier le queryset dans la méthode appropriée get_queryset( ) de la classe CategorieListView:

```
from django.db.models import Count

class CategorieListView(ListView):
        model = Categorie
        template_name = "monApp/list_categories.html"
        context_object_name = "categories"

        def get_queryset(self):
                # Annoter chaque catégorie avec le nombre de produits liés
                return Categorie.objects.annotate(nb_produits=Count('produit_categorie'))

        def get_context_data(self, **kwargs):
                context = super(CategorieListView, self).get_context_data(**kwargs)
                context['titremenu'] = "Liste de mes catégories"
                return context
```

Nous nous servons ici de la relation existante produit_categorie que nous avons déclaré dans notre modèle Produit :

```
categorie = models.ForeignKey(Categorie, on_delete=models.CASCADE, related_name="produit_categorie")
```

Quant au template list_categories.html, il devient ceci (je vous montre ici uniquement le tableau) :

```
<table class="table">
	<thead>
		<th>Id</th>
		<th>Nom catégorie</th>
		<th>Nombre de Produits</th>
	</thead>
	<tbody>
	{% for cat in categories %}
		<tr>
			<td><a href="{% url 'detailCategorie' cat.idCat %}">{{ cat.idCat }}</a></td>
			<td>{{ cat.nomCat }} </td>
			<td>{{ cat.nb_produits }} </td>
		</tr>
	{% endfor %}
	</tbody>
</table>
```

Regardons vite le résultat à l'adresse http://127.0.0.1:8000/monApp/categories/. Vous devriez avoir le nom de l'attribut agrémenté de sa valeur. Idéalement, nous devons retrouver ce même niveau d'informations dans la vue de détail d'une catégorie. Modifions la classe CategorieDetailView :

```
class CategorieDetailView(DetailView):
	model = Categorie
	template_name = "monApp/detail_categorie.html"
	context_object_name = "ctgr"

	def get_queryset(self):
		# Annoter chaque catégorie avec le nombre de produits liés
		return Categorie.objects.annotate(nb_produits=Count('produit_categorie'))

	def get_context_data(self, **kwargs):
		context = super(CategorieDetailView, self).get_context_data(**kwargs)
		context['titremenu'] = "Détail de la catégorie"
		return context
```

Comme précédemment, la variable nb_produits sera utilisée dans le template detail_categorie.html :

```
{% block contenu %}
        <h2>{{ ctgr.nomCat }} contenant {{ ctgr.nb_produits }} produit(s)</h2>

        <ul>
                <li>Référence categorie : {{ctgr.idCat }} </li>
                <li>Nom catégorie : {{ ctgr.nomCat }}</li>
        </ul>
        <a href="{% url 'listCategories' %}" class="btn btn-secondary mt-2">Retour</a>
        <a href="{% url 'modificationCategorie' ctgr.idCat %}" class="btn btn-warning mt-2">Modifier</a>
        <a href="{% url 'suppressionCategorie' ctgr.idCat %}" class="btn btn-danger mt-2">Supprimer</a>
{% endblock %}
```

Rendez vous à l'adresse http://127.0.0.1:8000/monApp/categorie/2/ pour voir le résultat. Vous pouvez très facilement ajouter la liste des produits liés à la catégorie grâce au related_name="produit_categorie" que nous avons mis dans le modèle Produit. Rajoutons les produits au contexte dans la vue de détail :

```
        def get_context_data(self, **kwargs):
                context = super(CategorieDetailView, self).get_context_data(**kwargs)
                context['titremenu'] = "Détail de la catégorie"
                context['produits'] = self.object.produit_categorie.all()
```

Dans le template detail_categorie.html, nous affichons les produits liés à la catégorie en dessous de celle-ci. Je ne sais pas vous, mais j'en profite pour glisser quelques notions de cards Bootstrap ici (il faut bien que nous nous en servons) pour une meilleure ergonomie visuelle:

```
        {% extends 'monApp/base.html' %}

        {% block contenu %}
        <div class="card mb-4">
           <div class="card-header bg-primary text-white">
                <h4>Détail de la catégorie</h4>
           </div>
           <div class="card-body">
                <h5 class="card-title">{{ ctgr.nomCat }}</h5>
                <ul class="list-unstyled">
                   <li><strong>Référence :</strong> {{ ctgr.idCat }}</li>
                   <li><strong>Nom catégorie :</strong> {{ ctgr.nomCat }}</li>
                </ul>
                <div class="car-footer text-center">
                        <a href="{% url 'listCategories' %}" class="btn btn-secondary mt-2">Retour</a>
                        <a href="{% url 'modificationCategorie' ctgr.idCat %}" class="btn btn-warning mt-2">Modifier</a>
                        <a href="{% url 'suppressionCategorie' ctgr.idCat %}" class="btn btn-danger mt-2">Supprimer</a>
                </div>
           </div>
        </div>
```

```
<div class="card mb-4">
    <div class="card-header bg-primary text-white">
        <h4>Cette catégorie contient <strong>{{ ctgr.nb_produits }}</strong> produit(s)</h4>
    </div>
    <div class="card-body">
        <div class="row">
            {% if produits %}
                {% for prdt in produits %}
                    <div class="col-sm-3">
                        <div class="card mb-2 text-center">
                            <div class="card-header bg-info text-white">
                                <h5 class="card-title">{{ prdt.intituleProd }}</h5>
                            </div>
                            <div class="card-body">
                                <a href="{% url 'detailProduit' prdt.refProd %}" class="btn btn-primary mt-
                                2">Détails</a>
                            </div>
                        </div>
                    </div>
                {% endfor %}
            {% else %}
                <p class="text-muted">Aucun produit trouvé pour cette catégorie.</p>
            {% endif %}
        </div>
    </div>
</div>
{% endblock %}
```

C'est à vous de jouer !

Fort(e)s de cette expérience, vous allez faire de même pour

- la vue qui listent les statuts : afficher le nombre de produits selon le statut
- la vue de détail d'un statut : la même information et la liste de produits associés

Et la jointure ?

Compliquons la suite. Nous allons maintenant affiché la valeur totale en euros des produits qui composent un rayon dans la vue qui listent les rayons. C'est le `related_name="contenir_rayon"` que nous avons mis sur la ForeignKey vers `Rayon` dans `Contenir` qui nous permet de retrouver toutes les lignes de `Contenir` pour un rayon donné. Modifiez la fonction `get_context_data()` de la vue `RayonListView` telle que :

```
def get_context_data(self, **kwargs):
    context = super(RayonListView, self).get_context_data(**kwargs)
    context['titremenu'] = "Liste de mes rayons"
    rayons_data = []
    for rayon in context['rayons']:
        total = 0
        for contenir in rayon.contenir_rayon.all():
            total += contenir.produit.prixUnitaireProd * contenir.Qte
        rayons_data.append({'rayon': rayon,'total_stock': total})
    context['rayons_data'] = rayons_data
    return context
```

Quant au template list_rayons.html, il devient ceci (je vous montre ici uniquement le tableau) :

```
<table class="table">
        <thead>
                <th>Id</th>
                <th>Nom rayon</th>
                <th>Valeur marchande du rayon</th>
        </thead>
        <tbody>
        {% for data in rayons_data %}
                <tr>
                        <td><a href="{% url 'detailRayon' data.rayon.idRayon %}">{{ data.rayon.idRayon }}</a></td>
                        <td>{{ data.rayon.nomRayon }} </td>
                        <td>{{ data.total_stock }} €</td>
                </tr>
        {% endfor %}
        </tbody>
</table>
```

De la même manière, nous pouvons afficher dans la vue de détail d'un rayon RayonDetailView la même information. Mais nous allons compléter cette vue de détail avec la liste des produits qui composent le rayon avec leur nombre et leur prix. Votre contexte de votre vue devient :

```
def get_context_data(self, **kwargs):
        context = super(RayonDetailView, self).get_context_data(**kwargs)
        context['titremenu'] = "Détail du rayon"

        produits_data = []
        total_rayon = 0
        total_nb_produit = 0

        for contenir in self.object.contenir_rayon.all():
                total_produit = contenir.produit.prixUnitaireProd * contenir.Qte
                produits_data.append({'produit': contenir.produit,
                                      'qte': contenir.Qte,
                                      'prix_unitaire': contenir.produit.prixUnitaireProd,
                                      'total_produit': total_produit}                    )
                total_rayon += total_produit
                total_nb_produit += contenir.Qte

        context['produits_data'] =  produits_data
        context['total_rayon'] = total_rayon
        context['total_nb_produit'] = total_nb_produit

        return context
```

Dans le template detail_rayon.html, la partie inférieure qui affiche les produits et leurs informations ressemble à ceci (je vous laisse faire la partie supérieure):

```
<div class="card mb-4">
   <div class="card-header bg-primary text-white">
        <h4>Ce rayon contient <strong> {{total_nb_produit}} </strong> produit(s) pour un total de
             <strong>{{total_rayon|floatformat:2 }} € </strong></h4>
   </div>
   <div class="card-body">
        <div class="row">
          {% if produits_data %}
             {% for data in produits_data %}
               <div class="col-sm-3">
                  <div class="card mb-2 text-center">
                    <div class="card-header bg-info text-white">
                        <h5 class="card-title">{{ data.produit.intituleProd }}</h5>
                    </div>
                    <div class="card-body">
                        <ul class="list-unstyled">
                          <li>Prix unitaire : {{data.prix_unitaire }} € </li>
                          <li>Quantité : {{data.qte }} </li>
                          <li>Total : {{ data.total_produit}} €</li>
                        </ul>
                        <a href="{% url 'detailProduit' data.produit.refProd %}"
                        class="btn btn-primary mt-2">Détails</a>
                    </div>
                  </div>
               </div>
             {% endfor %}
              </div>
          {% else %}
             <p class="text-muted">Aucun produit trouvé pour cette catégorie.</p>
          {% endif %}
        </div>
</div>
```

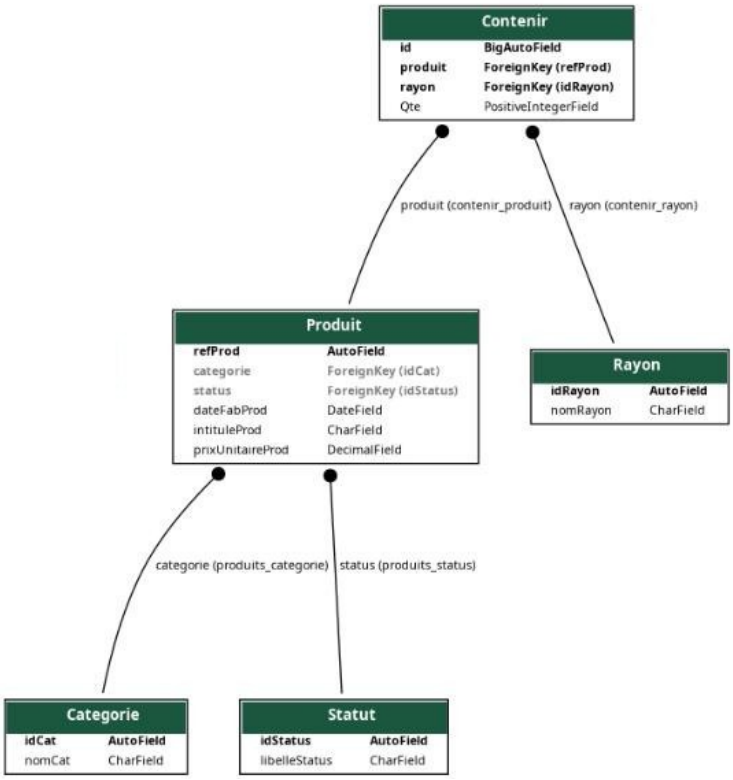
Et les performances dans tout ça ?

Contents êtes-vous ? Allez donc faire un tour du coté de la DJANGO Toolbar :

- dans le premier cas, à l'adresse http://127.0.0.1:8000/monApp/produits/, pas moins de 15 requêtes dont 7 requêtes similaires et dupliquées 2 fois ou 5 fois. Dans mon cas, j'ai 7 produits et nous allons chercher pour chacun d'entre eux la catégorie et le statut...
- dans le second cas, à l'adresse http://127.0.0.1:8000/monApp/rayons, pas moins de 13 requêtes dont 3 requêtes similaires !! Je vous laisse regarder en détail mais c'est une catastrophe ! Je n'ai que 3 rayons…

Inconscients vous êtes ? Une plus grosse volumétrie serait une catastrophe. Dans notre cas, il serait recommandé d'optimiser les requêtes avec select_related ou prefetch_related.
Qu'est ce que c'est ? Comment ça marche ?

Avant d'aller plus loin, nous avons besoin de nous rappeler notre modèle.



(Génération du MCD au passage)

- pip install django-extensions
- Dans le fichier monprojet/settings.py, ajoute django_extensions dans la liste des INSTALLED_APPS
- pip install pygraphviz
- pip install pydotplus
- ./manage.py graph_models -a -o mcd.png
- pip freeze > requirements.txt

Revenons à nos deux nouvelles notions.

- select_related est une méthode de l'ORM de DJANGO qui permet de faire des requêtes plus efficaces en préchargeant (ou "prefetching") des relations de type clé étrangère (ForeignKey) ou clé unique (OneToOneField). Cela permet de réduire le nombre de requêtes SQL effectuées lors de l'accès à des objets liés dans des relations de base de données. En d'autres termes, au lieu de faire une requête supplémentaire chaque fois que vous accédez à un objet lié via une clé étrangère, select_related effectue une jointure SQL pour obtenir toutes les données en une seule requête.

- prefetch_related est une méthode de l'ORM DJANGO qui permet de précharger (ou prefetch) des objets liés dans des relations de type Many-to-Many (ManyToManyField) ou des ForeignKey inversés. Il est utile lorsque les relations renvoient plusieurs objets ou lorsqu'on travaille avec des relations qui ne peuvent pas être résolues avec une jointure SQL simple, comme c'est le cas pour select_related. En d'autres termes, là où select_related fait une jointure SQL pour optimiser une relation de type ForeignKey ou OneToOneField, prefetch_related exécute des requêtes séparées pour les objets liés et minimise la duplication de requêtes en les récupérant à l'avance.

Donc si nous faisons le rapprochement avec notre modèle :

- pour aller de la table Produit à la table Statut, il faut utiliser select_related (ForeignKey classique = clé primaire de la table destination dans la table source)
- pour aller de la table Produit à la table Categorie, il faut utiliser select_related (ForeignKey classique = clé primaire de la table destination dans la table source)
- pour aller de la table Rayon à la table Produit en passant par Contenir, il faut utiliser prefetch_related + select_related combinés.

Soit. Appliquons donc ces deux premières règles à notre vue ProduitListView :

```
class ProduitListView(ListView):
    model = Produit
    template_name = "monApp/list_produits.html"
    context_object_name = "produits"

    def get_queryset(self):
        # Charge les catégories et les statuts en même temps
        return Produit.objects.select_related('categorie').select_related('statut')

    def get_context_data(self, **kwargs):
        context = super(ProduitListView, self).get_context_data(**kwargs)
        context['titremenu'] = "Liste de mes produits"
        return context
```

Les instructions select_related('categorie').select_related('statut') optimise la récupération de la categorie et du statut associés en une seule requête, pour chaque produit. En effet, notre bonne vieille amie DJANGO Toolbar nous indique plus que 3 requêtes au lieu de 15 !!!

Appliquons enfin la troisième règle à notre vue RayonListView :

```
class RayonListView(ListView):
    [...]

    def get_queryset(self):
        # Précharge tous les "contenir" de chaque rayon,
        # et en même temps le produit de chaque contenir
        return Rayon.objects.prefetch_related(
        Prefetch("contenir_rayon", queryset=Contenir.objects.select_related("produit"))
        )

    def get_context_data(self, **kwargs):
        [...]

        return context
```

D'une part, prefetch_related("contenir_rayon") charge toutes les lignes de la table Contenir liées aux rayons. D'autre part, select_related("produit"), pour chaque Contenir, joint directement le produit (pas besoin d'une requête supplémentaire par produit). Résultat : nous passons de 13 requêtes à 4 :

- Une pour tous les rayons.
- Une pour toutes les lignes de Contenir avec un JOIN sur Produit.

C'est à vous de jouer !

- Faites la même optimisation sur la vue de détail d'un rayon
- Vérifiez votre taux de couverture de tests ! Il y a peut être des choses à tester...

Barre de recherche

Nous allons ajouter une barre de recherche sur nos différentes vues listant les produits, les categories, les statuts et les rayons. Dans nos différents templates, list_produits.html, list_categories.html, list_statuts.html et list_rayons.html, il nous faut rajouter un bout de code. Personnellement, je l'ai positionné après le contenu du tableau, en lieu et place du paragraphe qui nous indique le nombre de produits :

```
<!-- Barre de recherche -->
<p class="lead mb-3 ml-0">Rechercher un élément...</p>
<form method="GET" action="">
        <input class="form-control" type="text" name="search" placeholder="Tapez ici votre recherche"
        value="{{ request.GET.search }}">
        <button type="submit" class="btn btn-success mt-2">Rechercher</button>
</form>
```

Le formulaire utilise la méthode GET pour soumettre la requête. Le champ de texte a pour nom search, qui correspond au paramètre GET que nous allons récupérer dans la vue.
La balise value="{{ request.GET. search }}" permet de garder le terme de recherche dans le champ après la soumission du formulaire.

Dans le fichier views.py, vous allez modifier la méthode get_queryset( ) de la vue ProduitListView pour afficher les produits filtrés par la recherche.

```
def get_queryset(self):
    # Surcouche pour filtrer les résultats en fonction de la recherche
    # Récupérer le terme de recherche depuis la requête GET
    query = self.request.GET.get('search')
    if query:
        return Produit.objects.filter(intituleProd__icontains=query).select_related('categorie').select_related('statut')

    # Si aucun terme de recherche, retourner tous les produits
    # Charge les catégories en même temps
    return Produit.objects.select_related('categorie').select_related('status')
```

La méthode get_queryset( ) permet de filtrer la liste des produits en fonction du terme de recherche (search) que l'on récupère à partir de l'URL (paramètre GET) avec self.request.GET.get('search'). Ici, nous utilisons name__icontains=query, ce qui filtre les produits dont le nom contient le terme de recherche (insensible à la casse).

C'est à vous de jouer !

Faites de même pour les vues CategorieListView, StatutListView et RayonListView.
Attention, toutefois, certaines adaptations seront nécessaires, notamment au niveau du filtre…
Enfin, pensez également à vos tests (méthode GET + paramètre search) et votre couverture de tests.

Les produits dans un rayon ou le retour de la jointure

Vous savez créer des produits et des rayons, bien. Mais saurez-vous ajouter un produit dans un rayon selon une certaine quantité… Vous voyez où je veux en venir ? Tout à fait ! Nous allons alimenter la table de jointure Contenir. Enfin, vous allez le faire mais je vous guide un peu :

☐ Créer un formulaire ContenirForm avec deux champs produit et Qte (selon votre modèle). Ne pas afficher le champ rayon car il sera défini automatiquement à partir du rayon courant.

☐ Créer la vue ContenirCreateView basée sur une classe, composé du modèle Contenir, de votre formulaire ContenirForm et du template monApp/create_contenir.html. Cette vue est utile pour gérer l'ajout d'un Contenir.

☐ Ajouter l'url `path('rayon/<pk>/contenir/', views.ContenirCreateView.as_view(), name='creationContenir')` dans le fichier urls.py

☐ Ajouter la méthode form_valid(self, form) à votre vue. Cette méthode :

- récupère les informations du formulaire
- vérifie si le produit existe déjà dans le rayon :
  - si oui, ajuste la quantité et sauvegarde
  - si non, crée la ligne dans Contenir avec les informations et sauvegarde
- redirige vers la vue de détail du rayon.

☐ Ajouter la méthode get_context_data(self, **kwargs) à votre vue pour ajouter dans le contexte l'objet correspondant au rayon courant identifié par <pk> :

```
context['rayon'] = Rayon.objects.get(pk=self.kwargs['pk'])
```

☐ Dans le template monApp/detail_rayon.html, ajouter un bouton dans la partie supérieure qui redirige vers le formulaire d'ajout d'un produit

```
<a href="{% url 'creationContenir' ryn.idRayon %}" class="btn btn-success">Ajouter un produit </a></h4>
```

☐ Créer le template monApp/create_contenir.html qui affiche votre formulaire

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit" class="btn btn-success">Valider</button>
    <a href="{% url 'detailRayon' rayon.idRayon %}" class="btn btn-secondary">Retour</a>
</form>
```

Je vous laisse gérer la mise à jour des quantités de produits en plus ou en moins au sein d'un rayon pour un produit donné (UpdateContenirView). Vous pouvez passer par un bouton Modifier ou bien deux boutons + et – qui appelle la même vue. Et quand la quantité du produit vaut zéro, il faut aussi penser bien sur à supprimer la ligne dans Contenir. (DeleteContenirView). Pour finir, testez et ajustez votre taux de couverture.